

Project Report: External Merge Sort for Large Employee Records

Name: *arwa mohey hamdy aboalia*

ID:23101371

1. Algorithm Selection Rationale and Implementation Details

Why External Merge Sort?

The primary objective of this project is to sort a dataset of 16,000 employee records intentionally distributed across 16 files (named `data_1.csv` to `data_16.csv`). This mirrors a real-world database scenario where datasets exceed the physical capacity of main memory (RAM). Standard sorting algorithms fail in such environments because they require the entire dataset to be resident in memory.

External Merge Sort was implemented because it efficiently handles out-of-memory sorting by utilising two distinct phases: a sorting phase (creating chunks) and a merging phase (multi-way merge), ensuring that memory consumption remains constant regardless of total dataset size.

Phase 1 — In-Memory Sorting (Chunking)

In this phase, the algorithm processes the 16 unsorted input files. For each file:

- The 1,000 employee records are loaded into a temporary buffer in RAM.
- The records are sorted in-place using Timsort (Python's `list.sort()`) with a complexity of $O(M \log M)$.
- The sorted buffer is written to a temporary chunk file (e.g., `sorted_chunk_1.csv`).
- The buffer is cleared to free memory for the next file.

Phase 2 — Multi-way External Merge Using a Min-Heap

This phase merges the 16 sorted chunks into a single final file. To maintain a low memory footprint, the algorithm uses a Min-Heap (Priority Queue):

- The first record from each of the 16 chunk files is loaded into the heap.
- The minimum element (based on `employee_id`) is popped and written to `final_sorted_employees.csv`.
- The algorithm immediately fetches the next record from the same source file and pushes it into the heap.

At no point does more than one row per source file reside in the heap, maintaining strict memory constraints.

2. Data Structure Design Choices

The Employee record is modelled using Python's `@dataclass` decorator:

```
@dataclass
class Employee:
    employee_id: int
    last_name: str
    first_name: str
    department: str
    salary: float
```

This design choice offers several concrete advantages over using a plain dictionary or tuple:

Feature	@dataclass	dict	namedtuple
Static type annotations	Yes	No	Partial
Auto-generated <code>__init__</code>	Yes	No	Yes
Mutable by default	Yes	Yes	No
IDE autocompletion / type checking	Yes	No	Partial
Readable <code>__repr__</code>	Yes	Yes	Yes

The field names defined in the dataclass are programmatically reused to derive the CSV header row, creating a single source of truth. If the schema changes (e.g., a new field is added), the output files adapt automatically without manual synchronisation — a deliberate defensive programming choice that eliminates schema drift.

3. Random Data Generation and File Handling

Data Generation Strategy

To simulate a raw database input, 16,000 unique IDs were generated and globally shuffled using `random.shuffle()`. Names were sampled randomly from predefined pools. Crucially, the Salary for each record was generated using a Uniform Distribution within the range [30,000.00 – 150,000.00], rounded to two decimal places to reflect realistic financial data.

Efficient File Handling

The project utilises `csv.DictReader` for structured data access. To manage 17 open file handles simultaneously during Phase 2, `contextlib.ExitStack` was employed to prevent resource leaks and ensure all files close safely upon completion or error. This provides the same safety guarantee as writing 17 individually nested with statements, but in a concise and maintainable form.

4. Time and Space Complexity Analysis

Let $N = 16,000$ (total records), $K = 16$ (chunks), and $M = 1,000$ (records per chunk). Note that $N = K \times M$.

Phase	Time Complexity	Space Complexity
Phase 1 (Sorting)	$O(N \log M)$	$O(M)$
Phase 2 (Merging)	$O(N \log K)$	$O(K)$
Overall	$O(N \log N)$	$O(M)$

Phase 1 derives its time complexity from applying Timsort — $O(M \log M)$ — to each of the K chunks, yielding $K \times O(M \log M) = O(N \log M)$. Only one chunk resides in RAM at a time, giving $O(M)$ space.

Phase 2 pushes and pops each of the N records exactly once from a heap of size K , costing $O(\log K)$ per operation and $O(N \log K)$ in total. The heap holds at most K records simultaneously, giving $O(K)$ space.

The combined time simplifies to $O(N \log M) + O(N \log K) = O(N * (\log M + \log K)) = O(N \log(M * K)) = O(N \log N)$, matching the information-theoretic lower bound for comparison-based sorting.

5. Evaluation of Correctness and Results

Correctness Verification

After execution, the following verification steps were performed on `final_sorted_employees.csv`:

- **Record Count:** Confirmed the output file contains exactly 16,000 records (plus header), ensuring no data loss during merging.
- **Monotonic Integrity:** Verified that `employee_id` values are monotonically increasing across the entire file ($ID[i] < ID[i+1]$), proving the multi-way merge logic is correct.

Performance Results

Metric	Execution Time
Data Generation	0.1209 seconds
Phase 1 (Sorting 16 chunks)	0.5749 seconds
Phase 2 (Multi-way Merge)	0.5192 seconds
Total Runtime	1.2177 seconds

Data Generation involves 16 sequential disk-write operations in addition to random number and string generation. Phase 1 is dominated by I/O cost — reading and writing 1,000 rows per file — since $M = 1,000$ makes the Timsort step extremely fast. Phase 2 processes all 16,000 records through the heap; because the heap holds only $K = 16$ elements, each heap operation costs $\log(16) = 4$ comparisons, making the merge computationally cheap.

6. Conclusion

The External Merge Sort implementation demonstrates that by partitioning data and using efficient priority-based merging, large-scale employee records can be sorted with minimal memory overhead. Sorting and merging 16,000 distributed records in 1.21 seconds practically validates the $O(N \log N)$ theoretical bound.

The use of a Min-Heap restricts memory consumption to exactly 16 records during the merge phase — $O(K)$ — proving that intelligent data structure selection can overcome severe hardware limitations. Furthermore, this project directly simulates the internal mechanics of real Relational Database Management Systems (RDBMS): when a database executes an ORDER BY clause on a table that exceeds buffer pool limits, it employs this exact External Merge Sort strategy, writing temporary segments to disk and merging them securely.

Ultimately, combining Python's memory management tools (ExitStack, dataclass) with robust algorithmic theory produces a scalable, correct, and production-grade data sorting pipeline.